Chess-Bot Engineering Roadmap

Master Plan • Phases 0 to 7

A full-stack, pedagogical roadmap for building an adaptive AI chess app with an automated blunder coach.

1. PROJECT GOAL

Build a modern, responsive web application where players of any skill level can challenge an adaptive, human-like AI bot and receive friendly, constructive post-game blunder coaching that accelerates improvement without confusing jargon.

2. NOVELTY & CORE DIFFERENTIATORS

1. Human-Like Bot: Most bots feel like either flawless calculators or erratic blunder machines. Chess-Bot models genuine human traits: psychological blind spots, time-management habits, and natural inaccuracy calibrated to the user's level.

2. Mistake Coach: Rather than dumping overwhelming evaluation graphs and complex 15-move variations, the coach isolates the player's **top 3 blunders**, tags each blunder with human concepts (e.g., *Hanging Piece, Missed Pin*), and explains why in plain, encouraging words.

3. TECH STACK & ARCHITECTURE RATIONALES

Technology	One-Line Rationale
Next.js + TypeScript (App Router)	Industry-standard React framework providing strict type safety, fast page routing, and clean project structure.
Tailwind CSS	Utility-first styling for rapid, polished, and fully responsive UI design without writing bulky custom CSS.
chess.js	Handles 100% of official chess rules, move validation, and game state transitions reliably out of the box.
react-chessboard	Pre-built, performant drag-and-drop chessboard component with smooth piece animations.
Custom Engine (TS + Web Worker)	Our own Minimax + Alpha-Beta AI logic running in a background thread to keep the browser UI silky smooth.
Stockfish (WASM)	Browser-native grandmaster engine used to accurately compute blunder thresholds and benchmark our custom bot.
LLM API (Later Phase)	Translates raw numeric evaluations into plain-English, supportive coaching explanations (optional & cost-efficient).
Supabase (Later Phase)	Scalable database and authentication service for user profiles, game history storage, and mistake tracking.
Vercel	Seamless, zero-configuration cloud hosting platform optimized specifically for Next.js applications.

4. PHASE-BY-PHASE ROADMAP

Phase 0: Project Setup & Architecture Foundations

Phase 0

Goal: *Establish a clean Next.js project with TypeScript, Tailwind CSS, core chess dependencies, and strict git hygiene.*

KEY STEPS

- Initialize Next.js project using App Router, TypeScript, and Tailwind CSS.
- Install core libraries: `chess.js` and `react-chessboard`.
- Set up clean folder architecture (`/components`, `/lib/engine`, `/workers`, `/types`, `/docs`).
- Configure strict `.gitignore` (ignoring `node_modules`, `.next`, `.env*`, and secrets).

DONE WHEN

- [ ] `npm run dev` starts without warnings or errors.
- [ ] Base page renders in browser and packages are verified.

Phase 1: Playable Board vs. Random Bot

Phase 1

Goal: *Provide a fully interactive, responsive board where the user can play full legal games against a basic random bot.*

KEY STEPS

- Connect `react-chessboard` UI to an active `chess.js` instance.
- Implement drag-and-drop legal move execution and turn management.
- Build a simple bot that selects a legal move uniformly at random.
- Implement game status banners: Check, Checkmate, Stalemate, and Draw.
- Add essential board controls: **New Game**, **Undo Move**, and **Flip Board**.

DONE WHEN

- [ ] Pieces drag and drop strictly following official chess rules.
- [ ] Random bot replies automatically on black's turn.
- [ ] Checkmate, check, and draw dialogs trigger accurately.
- [ ] New Game, Undo, and Flip buttons function reliably.

Phase 2: Bot v1 (Minimax Material Engine)

Phase 2

Goal: *Create our first calculating bot that values pieces and looks 2 moves ahead to win material and avoid simple traps.*

KEY STEPS

- Assign standard material weights: Pawn (100), Knight (300), Bishop (300), Rook (500), Queen (900), King (20,000).
- Build a board evaluation function (`lib/engine/evaluate.ts`) summing net material.
- Implement the recursive Minimax algorithm searching 2 plies deep (1 full round).
- Add a UI selector allowing the user to switch between Random Bot and Bot v1.

DONE WHEN

- [ ] Bot captures undefended (hanging) pieces whenever possible.
- [ ] Bot avoids moving valuable pieces into immediate capture.
- [ ] Move computation finishes in under 200ms per turn.

Phase 3: Smarter & Faster Bot (Alpha-Beta + Web Worker)

Phase 3

Goal: *Upgrade calculation depth and positional IQ with Alpha-Beta pruning, Piece-Square tables, and non-blocking Web Workers.*

KEY STEPS

- Implement **Alpha-Beta Pruning** to eliminate dead branches and reach depth 3–4.
- Incorporate **Piece-Square Tables (PST)** to reward center control, king safety, and active pieces.
- Add move ordering (prioritizing captures and checks) to trigger cutoffs early.
- Offload engine computation to a dedicated **Web Worker** to prevent UI thread freezing.
- Expose difficulty levels (Easy: depth 2; Medium: depth 3; Hard: depth 4).
- Benchmark depth and move accuracy against Stockfish WASM.

DONE WHEN

- ☐ Reaches depth 4 in less than 1 second per move.
- ☐ Chessboard UI remains silky smooth with zero input lag while bot calculates.
- ☐ Bot demonstrates clear opening development and easily defeats Bot v1.

Phase 4: Human-Like Bot (Adaptive & Organic)

Phase 4

Goal: *Give the bot organic human tendencies, realistic thinking delays, natural blind spots, and adaptive rating tuning.*

KEY STEPS

- Model human inaccuracy curves: introduce mild evaluation variance rather than choosing robotic #1 moves.
- Simulate human blind spots (e.g., long-range backward bishop diagonals, pawn-shield pins).
- Add realistic variable delays: instant moves for recaptures, longer pauses in complex positions.
- Implement dynamic adaptive difficulty that subtly adjusts engine strength to keep games engaging.

DONE WHEN

- ☐ Gameplay feels like a natural match against a human club player.
- ☐ Thinking times mirror realistic human reaction and contemplation speeds.
- ☐ Player enjoys close, competitive matches without frustrating engine walls.

Phase 5: Mistake Coach (Stockfish WASM + Explanations)

Phase 5

Goal: *Automatically identify the user's top 3 blunders after every game and explain them in plain, encouraging language.*

KEY STEPS

- Run Stockfish WASM in a background worker to evaluate each move's centipawn swing.
- Isolate the player's 3 worst mistakes (sharpest drop in win probability).
- Tag mistakes with intuitive conceptual labels: *Hanging Piece, Missed Win, Lost Tempo, Self-Pin*.
- Render an interactive review board highlighting the blunder square and the ideal move arrow.
- Generate clear, non-condescending coaching explanations (with optional lightweight LLM API).

DONE WHEN

- ☐ Game review card generates automatically within 5–10 seconds of game completion.
- ☐ Top 3 blunders are highlighted on an interactive timeline with visual board annotations.
- ☐ Each blunder displays a clear explanation of why it failed and what to look for next time.

Phase 6: Real Product (Auth, Game History & Blunder Puzzles)

Phase 6

Goal: *Turn the app into a full personal chess training platform with user login, game logs, and custom blunder puzzles.*

KEY STEPS

- Integrate Supabase Auth (Email + Google Sign-In).
- Persist completed game histories, moves (PGN), and blunder reports in Supabase.
- Build a personal dashboard showing win/loss trends and blunder category frequencies.
- Create a **"Blunder Practice Mode"** that converts the user's past mistakes into solvable puzzles.
- Optimize responsive touch layouts for mobile screens and configure Progressive Web App (PWA) manifest.

DONE WHEN

- ☐ Users can sign in, view past game analyses, and track blunder patterns over time.
- ☐ Blunder Practice mode generates interactive puzzles directly from user's historical games.
- ☐ App is touch-friendly and installable as a mobile PWA.

Phase 7: Launch, Polish & Community Testing

Phase 7

Goal: *Deploy to production on Vercel, conduct user testing with 5–10 friends, resolve UX issues, and publish documentation.*

KEY STEPS

- Deploy production build to Vercel and verify production environment variables.
- Conduct structured playtesting sessions with 5–10 friends across multiple devices.
- Address reported bugs, UI papercuts, and mobile layout edge cases.
- Author a polished, student-friendly `README.md` with architecture overview, screenshots, and setup steps.

DONE WHEN

- [ ] Web app is live on a public Vercel production URL with SSL.
- [ ] At least 5 real users have played matches and generated mistake reports.
- [ ] Project `README.md` is comprehensive with clear setup and gameplay screenshots.

Pedagogical Rules & Workflow Reminder: 1 phase at a time • Explanations before code • Code explained block-by-block with exact file paths • Private GitHub repo with `.gitignore` enforced • No push without user approval.